ATC'26 · HONG KONG · NOVEMBER 2026

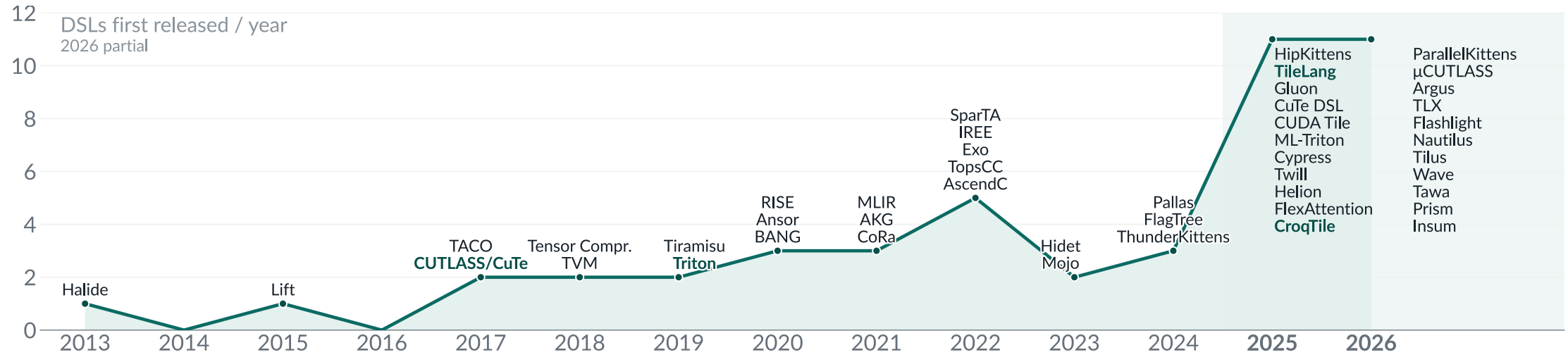
A Novel AI-Native GPU Kernel Programming Language for Autonomous Kernel Tuning

Heng Shi · Xiaofeng Guan · Sijie Wu · Enming Fan · Chenpeng Wu
Qiqi Gu · Jiahang Xie · Guoqing Bao · Jianguo Yao

Shanghai Enflame Intelligence & Shanghai Jiao Tong University

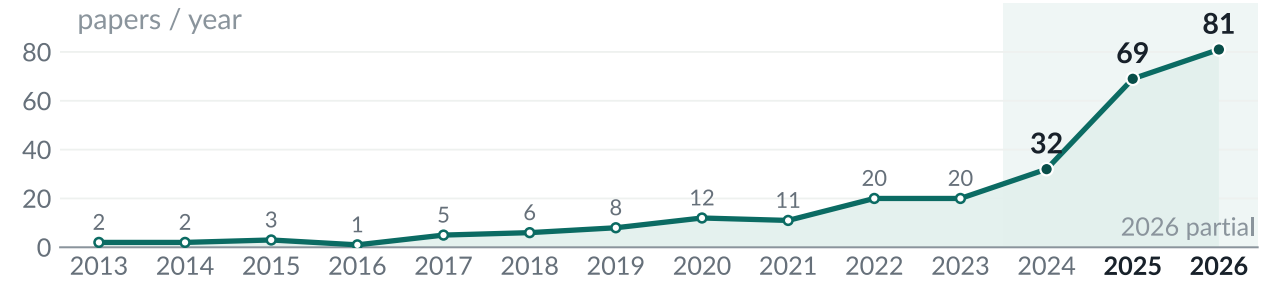
Operator work keeps growing

Model families converge. ***Kernel variants multiply.***



22/46 **DSLS IN 2025–26**

- **More programming substrates**
  46 DSLs in the survey corpus.
- **More operator work**
  276 optimization papers across 2013–26.



Why kernels multiply

1	**Operator variants**	MHA · MQA · GQA · MLA · sliding-window · block-sparse Attention keeps branching
2	**Hardware diversity**	NVIDIA · AMD MI300X · Ascend · TPU · Trainium One kernel, many targets
3	**New GPU instructions**	Hopper: WGMMA + TMA → Blackwell: tcgen05 + TMEM New instructions, new dataflows
4	**Operator fusion**	Conv + BN + ReLU → fusion changes the kernel boundary Eliminate intermediate traffic

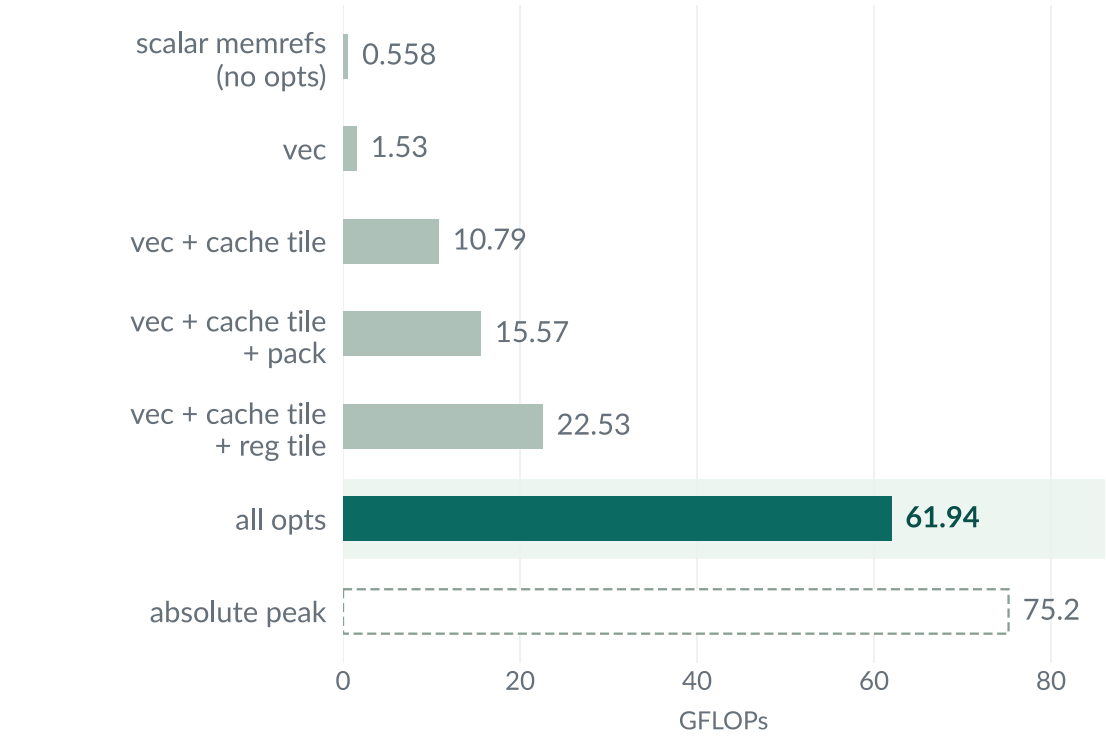
The maintenance burden multiplies. Operator variants × hardware targets × optimization choices.

Kernel optimizations interact

COUPLED PERFORMANCE DECISIONS

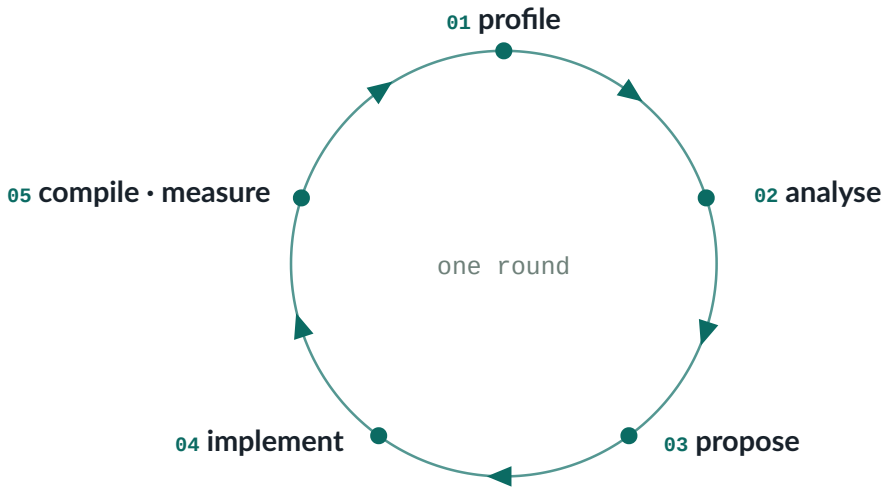
- **Locality**
  Tiling · data reuse · L2 bypass / cache policy
- **Data movement**
  Swizzling · vectorization · async copies
- **Execution overlap**
  Multibuffering · warp specialization · scheduling
- **Compute & cooperation**
  Unrolling · approximation · sparsity · fusion
  MMA layouts · low-precision MMA · clusters / DSMEM

Performance as optimizations combine



A larger tile improves reuse… …but consumes more shared memory, limiting stages and overlap.

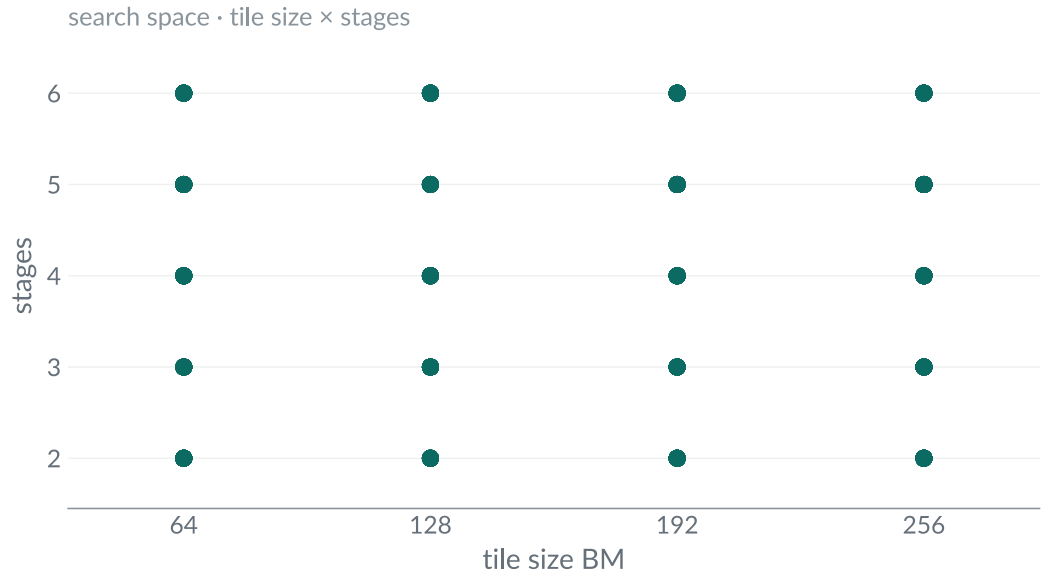
Bottleneck-driven tuning



1. Profile
2. Identify bottleneck
3. Choose structural edit
4. Implement
5. Compile · measure · keep / reject

- **Example: tensor cores waiting for data**
  Introduce a producer–consumer pipeline.
- **Expert effort in every round**
  Interpret counters, redesign the kernel, validate the edit.

Template search



`Tile size · stages · warps · swizzle`

- **Fixed skeleton**
  Choose the implementation structure up front.
- **Parameter search**
  Enumerate tile sizes, stages, warps and swizzles.
- **Filter, then benchmark**
  Reject resource violations; measure valid candidates.

Boundary: a new producer–consumer structure must first be exposed in the template.

Compiler transformations

data tiling / blocking	→ **`Loop tiling`**
data stationary	→ **`Cache staging`**
multibuffering & prefetch	→ **`Software pipelining`**
loop unroll	→ **`Unrolling`**
vectorisation	→ **`Vectorization`**
loop fusion	→ **`Fusion`**
instruction scheduling	→ **`Instruction scheduling`**

`MLIR · TVM · IREE`

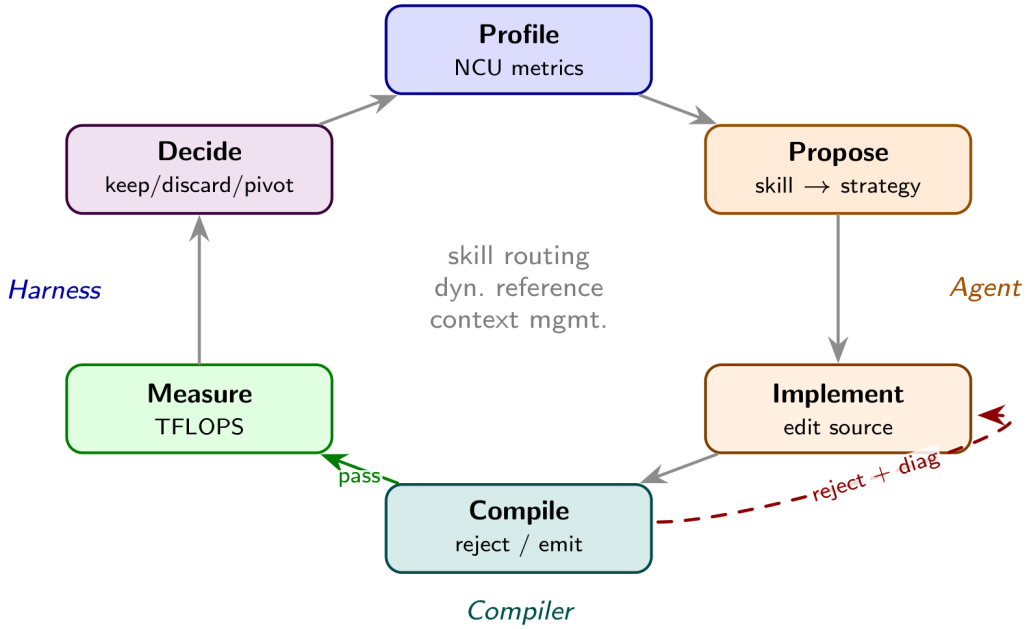
- **Schedule DSL**
  Express transformations in Halide / TVM TE.
- **Compiler passes**
  Encode legal rewrites over loops and dataflow.

New hardware, new structure: someone still needs to implement the corresponding transformations.

Agents rewrite the kernel

PROFILE → EDIT → EVALUATE

Full source edits · structural + parametric



WORKFLOW	Planning · search · refinement
EVALS	Correctness · speed · generalization
TOOLING	Profiler · compiler · benchmark Our focus: **language + compiler feedback** Make structural edits practical for the agent.
KNOWLEDGE	Skills · references · history

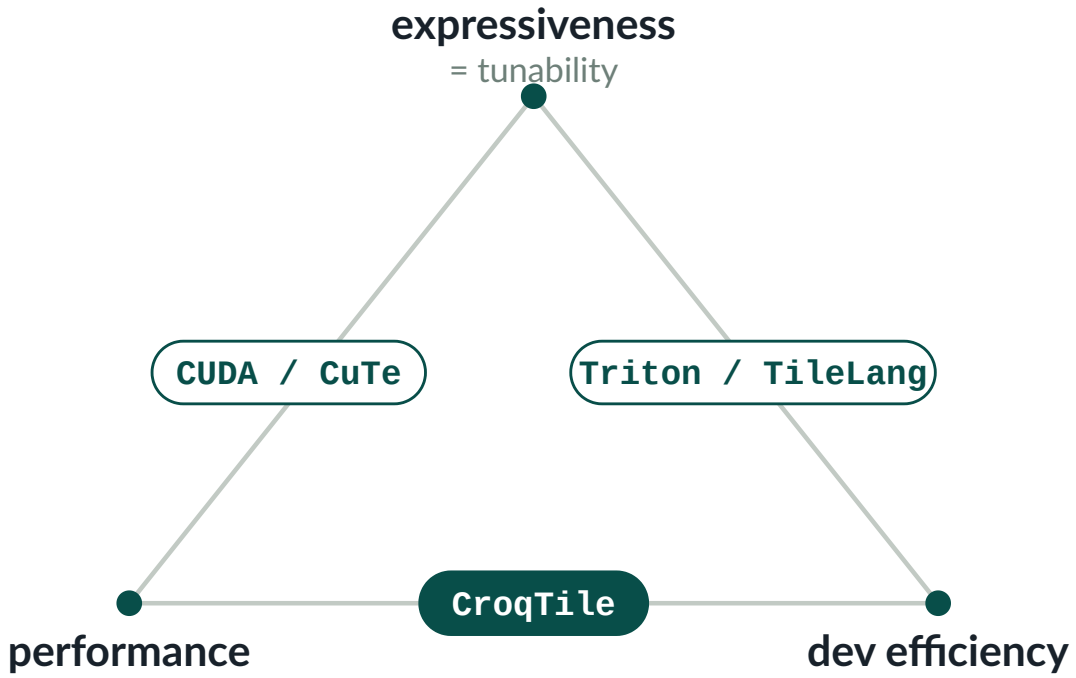
The agent can rewrite the structure. The programming language determines how costly each edit is.

Design for agent failures

FAILURE	REQUIREMENT
Context pressure Source + profiler output + debugging history	**Compact source** Compact programs leave room for prior attempts and measurements.
Coupled edits TMA: descriptor, swizzle, copy, synchronization	**Independent primitives** Give each performance decision an explicit, local edit site.
Opaque failures GPU hang or an error without a source location	**Early, actionable feedback** Reject early; report the constraint, edit site and repair.

Performance + cheap iteration

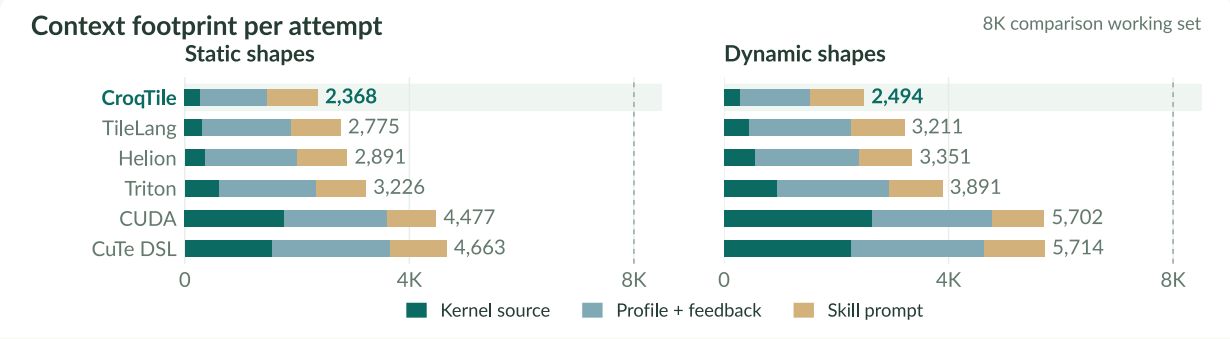
EXPRESSIVENESS · PERFORMANCE · ITERATION COST

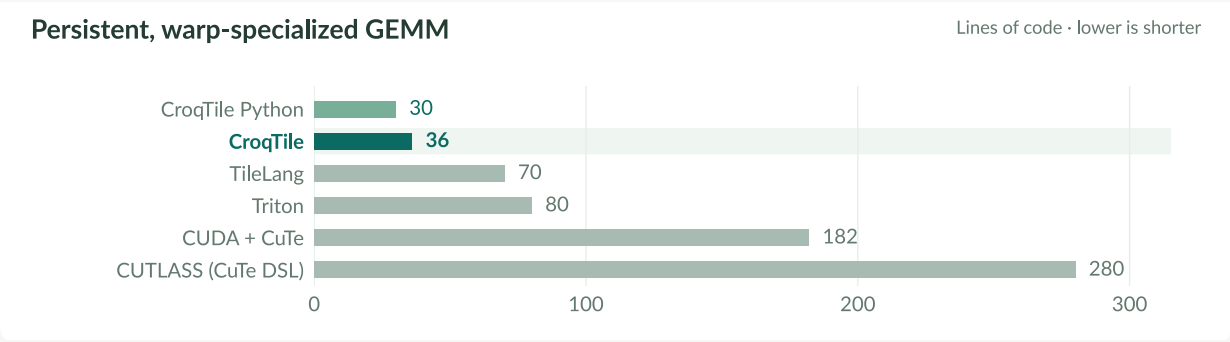


`CUDA / CuTe`	**Explicit control**	Long source · coupled edits
`Triton / TileLang`	**Compact tile operations**	Some structural controls hidden

The agent still chooses the design. Keep tiling, staging, warp specialization and data movement explicit—and compact to edit.

Compact syntax preserves context





Five constructs | Full EBNF

```
// 1 · Named shapes
shared f16[BM, BK] tile;
// 2 · Parallel binding
parallel p by 2 : group-4 { ... }
// 3 · Synchronization
shared event full[STAGES];
wait full[stage]; trigger full[stage];
// 4 · Data movement
tma.copy.swiz<128> src => dst;
// 5 · Matrix operations
mma.row.row mc, ma, mb;
```

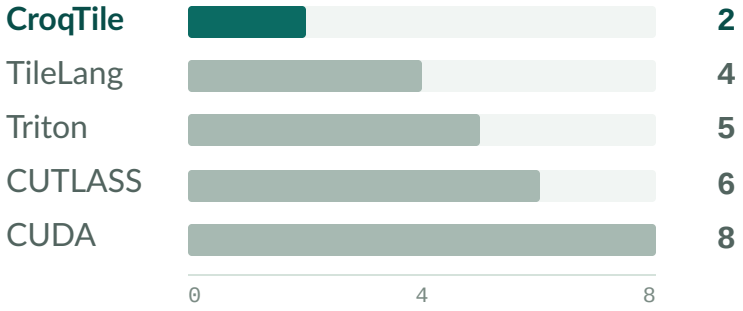
One statement per performance decision · illustrative syntax

- **36 vs. 182 lines**
  Same persistent, warp-specialized GEMM: CroqTile vs. CUDA + CuTe.
- **1.6× more room for tuning history**
  Source + feedback + skills: ≈2.37K vs. ≈4.48K tokens per static-shape attempt.

parallel: one construct, three levels

Parallelism primitives

Distinct constructs · lower is simpler



- **One binding, three levels**
  `block` · `group-4` · `thread`
- **Warp specialization**
  Split TMA producers and MMA consumers with `inthreads.async`.
- **Local structural edits**
  Change the parallel binding; keep the compute body.

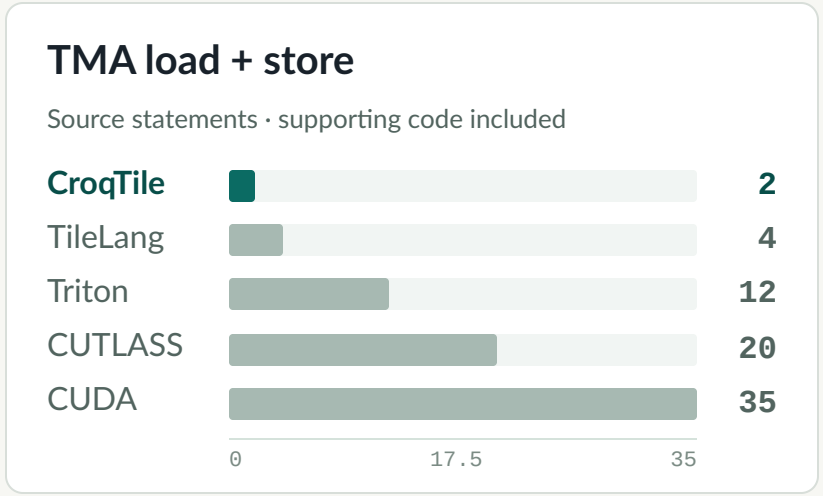
CroqTile | TileLang | CuTe / CUTLASS | CUDA

```
parallel {bm, bn}
  by [cdiv(M, WARP_M), cdiv(N, WARP_N)] : block {
  parallel p by 1 : group-4 {
    mma.row.row mc, ma, mb;
  }
  parallel {m, n}
    by [WARP_M, 2] : thread { ... }
}

// Producer–consumer warp specialization
parallel p1 by 2 : group-4 {
  inthreads.async (p1 == 0) { /* TMA producer */ }
  inthreads.async (p1 == 1) { /* MMA consumer */ }
}
```

Parallel binding · execution level · role assignment

tma: one statement per transfer



- **One statement per transfer**
  Tile source → destination; explicit load and store.
- **Keep the performance choice**
  `.swiz<128>` selects the swizzle; `.zfill` handles tails.
- **Derive the machinery**
  The compiler generates the descriptor and transfer details.

CroqTile | Triton | TileLang | CuTe / CUTLASS | CUDA

```
// LOAD · global → shared
tma.copy.swiz<128>
  lhs.subspan(WARP_M, TILE_K)
     .at(bm, iv_k) => lhs_s;

// STORE · shared → global
tma.copy out_s =>
  output.subspan(WARP_M, WARP_N)
        .at(bm, bn);
```

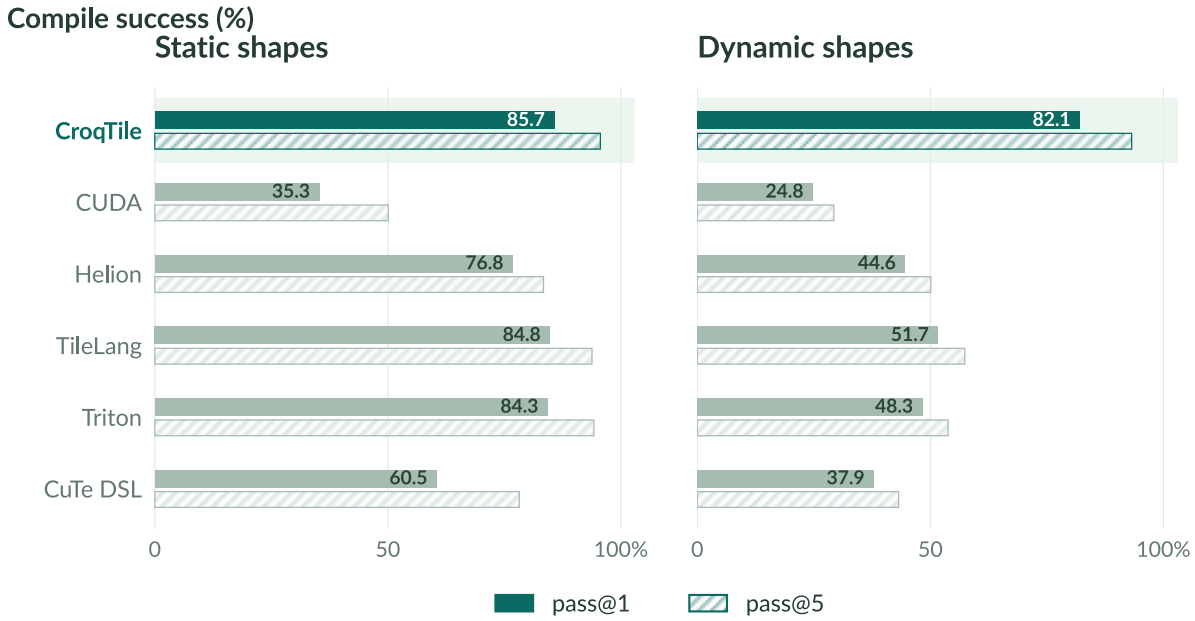
Two transfer statements · wrapped for readability

Agent chooses
Tile · swizzle · async policy

Compiler derives
Descriptor · transfer machinery

Compile success across shapes

PASS@1 / PASS@5 · STATIC + DYNAMIC



Same agent, hardware and budget. **Compile success** leaves more trials for performance tuning.

1 Static shapes

85.7% pass@1 · **95.6%** pass@5

Static shapes, with warp-level controls exposed.

2 Dynamic shapes

82.1% pass@1 · Triton **48%**

Dynamic shapes retain high compile success.

3 Type system

Symbolic shapes + compile-time inference.

Fewer manual relationships to keep consistent.

Diagnostics that guide repair

`compile_result` — `REJECTED · stg3, 128×128`

<constraint>

DMA dim-0 mismatch

```
from.d0 == to.d0
```

<observed>

```
from.d0 = N / cdiv(N, 128)
to.d0 = 128
```

Source and destination dimensions must match.

<location>

spmm.co · line 58

```
tma.copy.async<full[stg]>.swiz<128>
rhs.subspan(WN,TK).at(bn,k)
=> rhs_s.subspan(WN,TK).at(stg,0);
```

<repair>

Append `.zfill` to zero-fill the tail tile.

When N % 128 ≠ 0, the source tile is smaller.

Alternative: pad tile_n to cdiv(N,128) × 128.

`Constraint → evidence → source location → concrete repair`

1 Symbolic engine

Check shape compatibility and resource limits **before launch**.

2 Runtime assertions

Check constraints that depend on runtime values.

3 XML-tagged messages

Return **machine-readable fields** to the tuning harness.

4 Root cause + fix

Point to the failing statement and suggest the **next edit**.

Prune before GPU execution

SPARSE GEMM · CONFIGURATION PRUNING

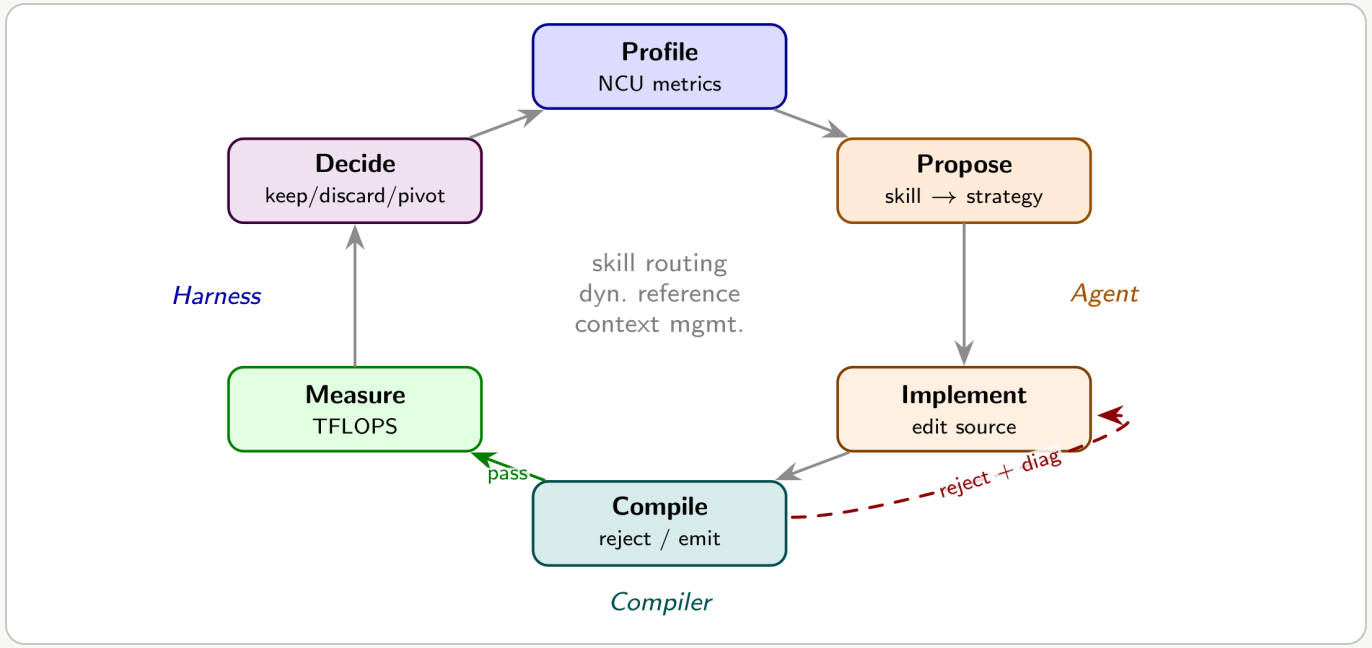


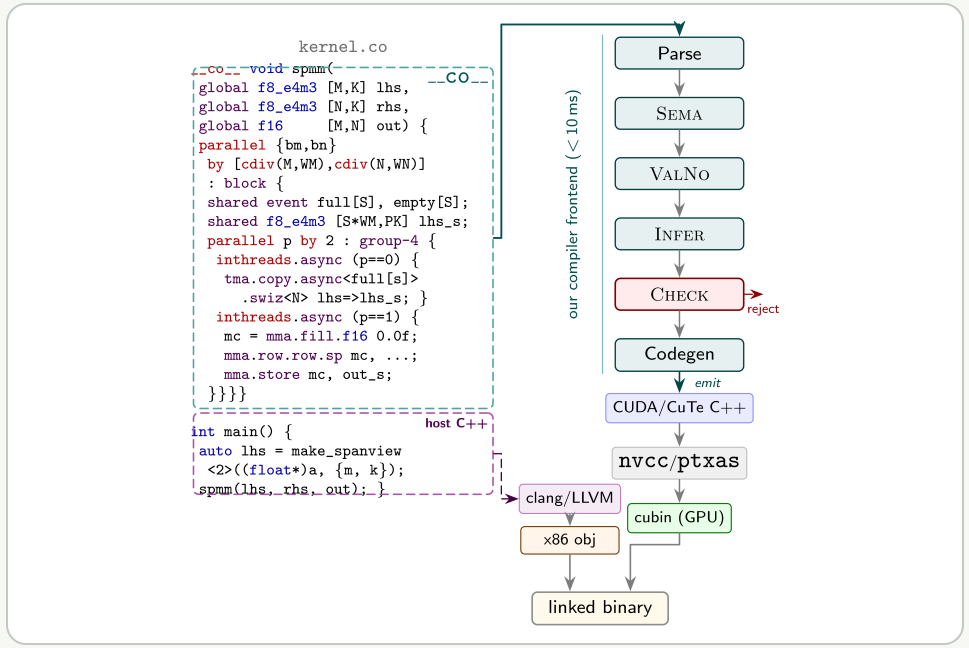
- **353 checks across 7 modules**
  Shapes · registers · shared memory · synchronization
- **480 candidates → 102 viable**
  79% pruned before GPU execution; mostly register and shared-memory limits.
- **Feedback changes the search**
  A local repair saves budget for the next performance candidate.

−14.9% throughput

With generic errors in place of structured feedback
Sparse GEMM ablation

Close the autonomous loop





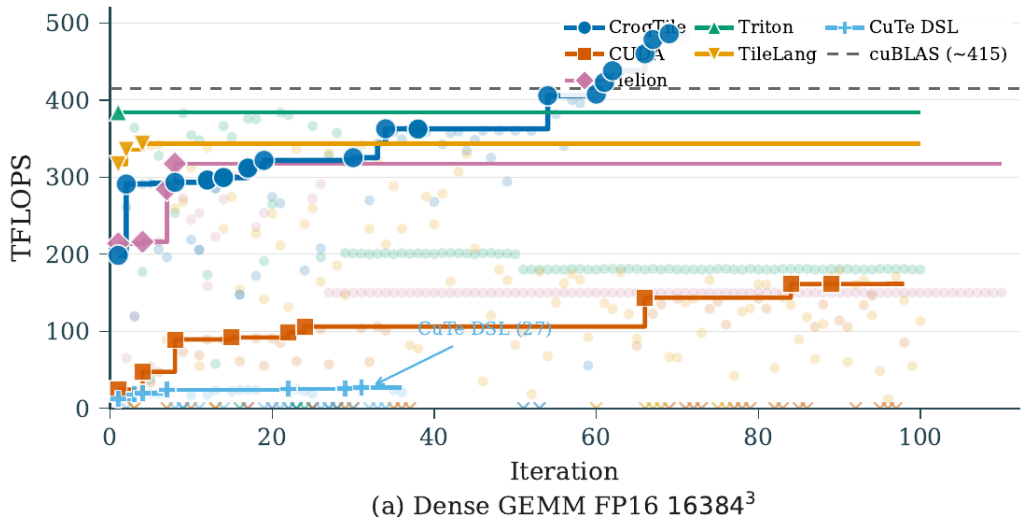
- **One agent, six phases**
  Profile → propose → implement → compile → measure → decide.
- **Skills + references**
  Language rules and bottleneck-specific ideas guide the next edit.
- **Verify before measuring**
  Compile rejection returns to implementation; valid kernels are checked and timed.

1.23× vendor libraries Geometric-mean speedup across evaluated workloads with vendor references.

Tuning beyond cuBLAS

Dense FP16 GEMM

M = N = K = 16,384

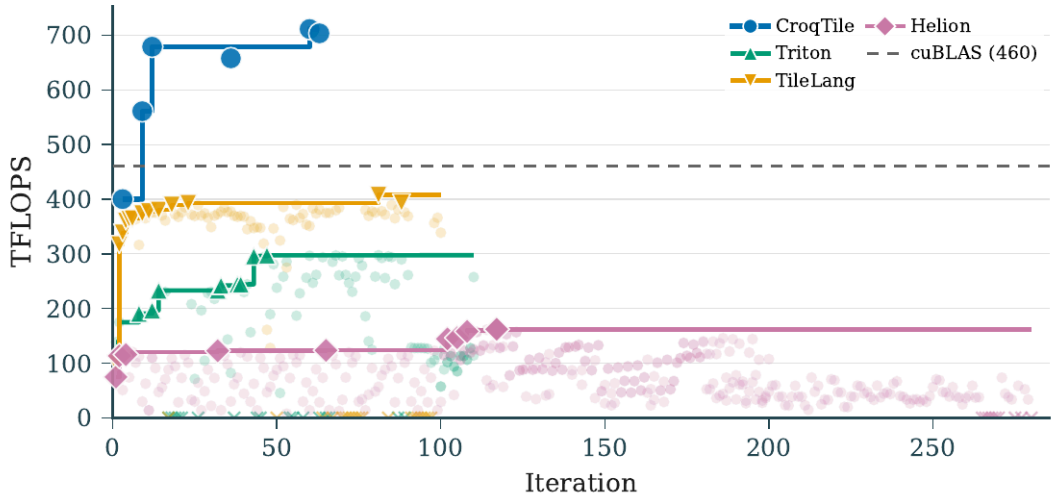


(a) Dense GEMM FP16 16384^3

Improves steadily, reaches cuBLAS, then exceeds it.

Block-scale E4M3 GEMM

M = N = K = 8,192



(b) Block-scale GEMM E4M3 8192^3

Crosses the vendor baseline early and continues improving.

- **Read the curves**
  Solid lines: running-best TFLOPS; dashed line: cuBLAS.

- **Shared evaluation protocol**
  H800 PCIe · Claude Sonnet 4.6 · same agent and per-shape time cap.

Compact programs. Actionable feedback.



CroqTile is open source

Scan · CroqTile repo

OPEN SOURCE · FIVE ACCEPTED WORKS

SC'26 · Structured Sparse GEMM on Hopper via Analytical Modeling

SC'26 · Do We Need Tensor Cores for Stencil Computations? **Best Paper Nominee**

ATC'26 · An AI-Native GPU Kernel Programming Language for Autonomous Tuning

ATC'26 · SptcAttn: Long-Context Attention with Diagonal-Stripes Sparse Tensor Core

CGO'27 · Signal-Aware Liveness Analysis for Shared Memory in Warp-Specialized Kernels

Thank you · Questions? Expose performance decisions. Preserve tuning history. Make every attempt informative.